Alert Notification Emails Using PowerShell

IMPORTANT NOTES - You need to have Outbound TCP port 587 open on the server or Firewall in order to send emails from the server. Also make sure both the "Queue SQL Agent Alert" and "Send Alert Emails Using Powershell" SQL jobs are always enabled and running otherwise none of this will work fully or in some cases at all.

You may end up to a point where Database Mail will not work correctly, or you would rather handle alert emailing using other methods rather than SQL itself to send notifications. Whatever the case, you can optionally leverage PowerShell to send alert email notifications if so desire. While there are numerous ways to accomplish this. I will show a way I utilized (and made it work) and you can tweak it how you like. The basic setup of how this all works is as follows...

Alert/Monitor calls Stored Procedure -> Stored Procedure takes alert details and creates a row in the email queue table in Pending status awaiting to have email sent -> SQL Job runs PowerShell script which goes out to Mail Queue table and grabs any Pending messages and creates an email and send it out -> PowerShell script marks sent items as Processed on table to prevent duplicate processing

Step 1 - Setup The PowerShell Script

First you will want to create a credential file to store login credentials. To do this you NEED to log in to the computer hosting SQL Server as the same account that runs the SQL Agent Service.

Once logged in, run the following commands in PowerShell, you can modify the file location and name as needed.

```
$cred = Get-Credential
$cred | Export-Clixml -Path "C:\yourfolder\smtp-cred.xml"
```

Once that file is created, you will enter the filename and location into the $Credential field of your PowerShell script below.

Create a new PowerShell script on the system the SQL server is installed on using whatever tool you want be it notepad, ISE, etc. and you can name and place it anywhere you like (make sure to take note of the file path and file name as we'll be using them here in a bit. Enter the following in the file and save...

```
param()

# ==============================
# SQL settings
# ==============================
$SqlServer = "sqlservername"
$Database  = "msdb"

# ==============================
# SMTP settings
# ==============================
$SmtpServer  = "smtp.office365.com"
$SmtpPort    = 587
$FromAddress = "emailaddress"

# ==============================
# Load credential from file
# ==============================
$Credential = Import-Clixml -Path "C:\yourfolder\smtp-cred.xml"

function New-SqlConnection {
   $connectionString = "Server=$SqlServer;Database=$Database;Integrated Security=True;"
   $connection = New-Object System.Data.SqlClient.SqlConnection $connectionString
   return $connection
}

function Invoke-SqlScalar {
   param([string]$Query)

   $connection = New-SqlConnection
   $command = $connection.CreateCommand()
   $command.CommandText = $Query
   $command.CommandTimeout = 60

   try {
```

```
    try {
        $connection.Open()
        return $command.ExecuteScalar()
    }
    finally {
        $connection.Close()
        $connection.Dispose()
    }
}

function Invoke-SqlNonQuery {
    param([string]$Query)

    $connection = New-SqlConnection
    $command = $connection.CreateCommand()
    $command.CommandText = $Query
    $command.CommandTimeout = 60

    try {
        $connection.Open()
        [void]$command.ExecuteNonQuery()
    }
    finally {
        $connection.Close()
        $connection.Dispose()
    }
}

function Invoke-SqlReaderSingleRow {
    param([string]$Query)

    $connection = New-SqlConnection
    $command = $connection.CreateCommand()
    $command.CommandText = $Query
    $command.CommandTimeout = 60

    try {
        $connection.Open()
        $reader = $command.ExecuteReader()

        if ($reader.Read()) {
            $row = @{
                AlertID      = $reader["AlertID"]
                EmailTo      = $reader["EmailTo"]
                EmailSubject = $reader["EmailSubject"]
                EmailBody    = $reader["EmailBody"]
            }
            $reader.Close()
            return $row
        }
        else {
            $reader.Close()
            return $null
        }
    }
    finally {
        $connection.Close()
        $connection.Dispose()
    }
}

try {
    $serverName = Invoke-SqlScalar "SELECT @@SERVERNAME;"
    $pendingCount = Invoke-SqlScalar "SELECT COUNT(*) FROM dbo.SQLAlertEmailQueue WHERE Status = 'Pending';"

    Write-Output "Connected to: $serverName"
    Write-Output "Pending rows found: $pendingCount"
```

```
    Write-Output "Pending rows found: $pendingCount"

    if ([int]$pendingCount -le 0) {
        Write-Output "No pending alert emails found."
        exit 0
    }

    $row = Invoke-SqlReaderSingleRow @"
SELECT TOP (1)
    AlertID,
    EmailTo,
    EmailSubject,
    EmailBody
FROM dbo.SQLAlertEmailQueue
WHERE Status = 'Pending'
ORDER BY AlertID ASC;
"@

    if ($null -eq $row) {
        Write-Error "Pending count was greater than 0, but no row could be read."
        exit 1
    }

    $AlertID = [int]$row.AlertID
    $EmailTo = [string]$row.EmailTo
    $Subject = [string]$row.EmailSubject
    $Body    = [string]$row.EmailBody

    Write-Output "Picked AlertID $AlertID for $EmailTo"

    Invoke-SqlNonQuery @"
UPDATE dbo.SQLAlertEmailQueue
SET Status = 'Processing'
WHERE AlertID = $AlertID
  AND Status = 'Pending';
"@

    Send-MailMessage `
        -From $FromAddress `
        -To $EmailTo `
        -Subject $Subject `
        -Body $Body `
        -SmtpServer $SmtpServer `
        -Port $SmtpPort `
        -UseSsl `
        -Credential $Credential `
        -ErrorAction Stop

    Invoke-SqlNonQuery @"
UPDATE dbo.SQLAlertEmailQueue
SET
    Status = 'Processed',
    ProcessedDate = GETDATE()
WHERE AlertID = $AlertID;
"@

    Write-Output "Email sent successfully for AlertID $AlertID"
    exit 0
}
catch {
    $msg = $_.Exception.Message.Replace("'", "''")
    Write-Error "PowerShell email job failed: $msg"

    if ($AlertID) {
        try {
            Invoke-SqlNonQuery @"
UPDATE dbo.SQLAlertEmailQueue
```

```
UPDATE dbo.SQLAlertEmailQueue
SET
   Status = 'Failed',
   ProcessedDate = GETDATE()
WHERE AlertID = $AlertID;
"@
      }
      catch {
         Write-Error "Failed to mark AlertID $AlertID as Failed: $($_.Exception.Message)"
      }
   }

   exit 1
}
```

You'll want to update **$SqlServer** to the name of the computer that has your SQL Server installed. You'll also want to set the **$FromAddress** to whichever email account used for exchange login and sending the emails. This script is using the Office 365 server for connection, but you can modify the **$SmtpServer** to an on-prem Exchange server or internal relay by changing the address.

Step 2 - Create The Mail Queue Table

Next you will want to create the Mail Queue table, this will be where pending alert messages are sent to be processed and emailed using the SQL Job/Powershell script.

```
USE [msdb];
GO

IF OBJECT_ID('dbo.SQLAlertEmailQueue', 'U') IS NULL
BEGIN
   CREATE TABLE dbo.SQLAlertEmailQueue
   (
      AlertID      INT IDENTITY(1,1) NOT NULL
         CONSTRAINT PK_SQLAlertEmailQueue PRIMARY KEY,
      EmailTo      NVARCHAR(500) NOT NULL,
      EmailSubject  NVARCHAR(500) NOT NULL,
      EmailBody     NVARCHAR(MAX) NOT NULL,
      CreatedDate   DATETIME NOT NULL
         CONSTRAINT DF_SQLAlertEmailQueue_CreatedDate DEFAULT (GETDATE()),
      ProcessedDate DATETIME NULL,
      Status       VARCHAR(20) NOT NULL
         CONSTRAINT DF_SQLAlertEmailQueue_Status DEFAULT ('Pending')
   );
END
GO
```

Step 3 - Create Table Index

After you create the table, it is best to create an Index on the queue table for faster lookups for Pending alert messages waiting to be sent.

```
USE [msdb];
GO

IF NOT EXISTS
(
   SELECT 1
   FROM sys.indexes
   WHERE object_id = OBJECT_ID('dbo.SQLAlertEmailQueue')
     AND name = 'IX_SQLAlertEmailQueue_Status_AlertID'
)
BEGIN
   CREATE INDEX IX_SQLAlertEmailQueue_Status_AlertID
   ON dbo.SQLAlertEmailQueue (Status, AlertID);
END
GO
```

Step 4 - Create The Stored Procedure

Then you will create a Stored Procedure, this is what will be called by your SQL Alerts and Monitors to create a row in the Mail Queue table with the Creation Date and Time of alert, Email Subject and Date. It then sets the row to a status of Pending, where it will be awaiting for processing.

For the highlighted portion, you can change the destination email address to a email account, shared mailbox or distribution group of your choosing.

```
USE [master];
GO

CREATE OR ALTER PROCEDURE dbo.usp_SendAlertEmail
   @Subject NVARCHAR(500),
   @Body NVARCHAR(MAX),
   @To NVARCHAR(500) = N'destinationemailaddress',
   @ThrottleMinutes INT = 10
AS
BEGIN
   SET NOCOUNT ON;

   SET @Subject = LEFT(ISNULL(@Subject, N'(No Subject)'), 500);
   SET @Body = ISNULL(@Body, N'');

   IF EXISTS
   (
      SELECT 1
      FROM msdb.dbo.SQLAlertEmailQueue
      WHERE EmailSubject = @Subject
        AND CreatedDate >= DATEADD(MINUTE, -@ThrottleMinutes, GETDATE())
   )
   BEGIN
      RETURN;
   END;

   INSERT INTO msdb.dbo.SQLAlertEmailQueue
   (
      EmailTo,
      EmailSubject,
      EmailBody
   )
   VALUES
   (
      @To,
      @Subject,
      @Body
   );
END;
GO
```

Step 5 - Create SQL Job To Run PowerShell Script & Cleanup

The next step is to setup a SQL job that runs your PowerShell script every 1 minute. What this script does is it will query the Mail Queue table for any rows (aka alert messages) that have a status of Pending, it then takes the details from this row to construct an alert email to be sent. It then sends out the alert email, then goes back to the table and marks the sent message as Processed.

This job also has a second step in which it runs a cleanup on the Mail Queue table and deletes anything older than 30 days.

The highlighted items below can be changed if needed. First item is the SQL Email Operator, which you can change to whatever SQL Server Agent Operator is setup on your server. Second is the location and file name on the computer of the PowerShell script you created above.

```
USE [msdb];
GO

DECLARE @JobName SYSNAME = N'Send Alert Emails Using Powershell';
DECLARE @ScheduleName SYSNAME = N'Send Alert Emails Every Minute';
```

```
DECLARE @ScheduleName SYSNAME = N'Send Alert Emails Every Minute';
DECLARE @JobId UNIQUEIDENTIFIER;

-----------------------------------------------------------------------------
-- Create job if it does not exist
-----------------------------------------------------------------------------
IF NOT EXISTS
(
   SELECT 1
   FROM dbo.sysjobs
   WHERE name = @JobName
)
BEGIN
   EXEC dbo.sp_add_job
      @job_name = @JobName,
      @enabled = 1,
      @description = N'DO NOT DISABLE/DELETE - An on-demand job SQL uses to send alert emails for SQL issues. This is used in place of
Database Mail.',
      @start_step_id = 1,
      @owner_login_name = N'sa',
      @notify_level_email = 2,  -- On failure
      @notify_email_operator_name = N'sqlemailoperator',
      @job_id = @JobId OUTPUT;
END
ELSE
BEGIN
   SELECT @JobId = job_id
   FROM dbo.sysjobs
   WHERE name = @JobName;

   EXEC dbo.sp_update_job
      @job_name = @JobName,
      @enabled = 1,
      @description = N'DO NOT DISABLE/DELETE - An on-demand job SQL uses to send alert emails for SQL issues. This is used in place of
Database Mail.',
      @notify_level_email = 2,
      @notify_email_operator_name = N'sqlemailoperator';
END
GO

-----------------------------------------------------------------------------
-- Add or update Step 1: Execute Powershell Script
-----------------------------------------------------------------------------
IF NOT EXISTS
(
   SELECT 1
   FROM dbo.sysjobsteps s
   INNER JOIN dbo.sysjobs j
      ON s.job_id = j.job_id
   WHERE j.name = N'Send Alert Emails Using Powershell'
    AND s.step_id = 1
)
BEGIN
   EXEC dbo.sp_add_jobstep
      @job_name = N'Send Alert Emails Using Powershell',
      @step_id = 1,
      @step_name = N'Execute Powershell Script',
      @subsystem = N'CmdExec',
      @command = N'powershell.exe -ExecutionPolicy Bypass -NoProfile -File "C:\yourfolder\yourpowershellscript.ps1"',
      @on_success_action = 3,   -- Go to next step
      @on_fail_action = 2;      -- Quit with failure
END
ELSE
BEGIN
   EXEC dbo.sp_update_jobstep
      @job_name = N'Send Alert Emails Using Powershell',
      @step_id = 1,
```

```
        @step_id = 1,
        @step_name = N'Execute Powershell Script',
        @subsystem = N'CmdExec',
        @command = N'powershell.exe -ExecutionPolicy Bypass -NoProfile -File "C:\yourfolder\yourpowershellscript.ps1"',
        @on_success_action = 3,
        @on_fail_action = 2;
END
GO

------------------------------------------------------------------------
-- Add or update Step 2: Cleanup Old Email Queue Records
------------------------------------------------------------------------
IF NOT EXISTS
(
    SELECT 1
    FROM dbo.sysjobsteps s
    INNER JOIN dbo.sysjobs j
        ON s.job_id = j.job_id
    WHERE j.name = N'Send Alert Emails Using Powershell'
      AND s.step_id = 2
)
BEGIN
    EXEC dbo.sp_add_jobstep
        @job_name = N'Send Alert Emails Using Powershell',
        @step_id = 2,
        @step_name = N'Cleanup Old Email Queue Records',
        @subsystem = N'TSQL',
        @database_name = N'msdb',
        @command = N'
DELETE FROM dbo.SQLAlertEmailQueue
WHERE CreatedDate < DATEADD(DAY, -30, GETDATE());
',
        @on_success_action = 1,   -- Quit with success
        @on_fail_action = 2;      -- Quit with failure
END
ELSE
BEGIN
    EXEC dbo.sp_update_jobstep
        @job_name = N'Send Alert Emails Using Powershell',
        @step_id = 2,
        @step_name = N'Cleanup Old Email Queue Records',
        @subsystem = N'TSQL',
        @database_name = N'msdb',
        @command = N'
DELETE FROM dbo.SQLAlertEmailQueue
WHERE CreatedDate < DATEADD(DAY, -30, GETDATE());
',
        @on_success_action = 1,
        @on_fail_action = 2;
END
GO

------------------------------------------------------------------------
-- Ensure job is attached to local server
------------------------------------------------------------------------
IF NOT EXISTS
(
    SELECT 1
    FROM dbo.sysjobservers js
    INNER JOIN dbo.sysjobs j
        ON js.job_id = j.job_id
    WHERE j.name = N'Send Alert Emails Using Powershell'
)
BEGIN
    EXEC dbo.sp_add_jobserver
        @job_name = N'Send Alert Emails Using Powershell',
        @server_name = N'(LOCAL)';
```

```
        @server_name = N'(LOCAL)';
END
GO

-------------------------------------------------------------------------------
-- Create schedule if it does not exist
-------------------------------------------------------------------------------
IF NOT EXISTS
(
    SELECT 1
    FROM dbo.sysschedules
    WHERE name = N'Send Alert Emails Every Minute'
)
BEGIN
    EXEC dbo.sp_add_schedule
        @schedule_name = N'Send Alert Emails Every Minute',
        @enabled = 1,
        @freq_type = 4,             -- Daily
        @freq_interval = 1,
        @freq_subday_type = 4,      -- Minutes
        @freq_subday_interval = 1,
        @active_start_time = 0;
END
GO

-------------------------------------------------------------------------------
-- Attach schedule to job if not already attached
-------------------------------------------------------------------------------
IF NOT EXISTS
(
    SELECT 1
    FROM dbo.sysjobschedules js
    INNER JOIN dbo.sysjobs j
        ON js.job_id = j.job_id
    INNER JOIN dbo.sysschedules s
        ON js.schedule_id = s.schedule_id
    WHERE j.name = N'Send Alert Emails Using Powershell'
      AND s.name = N'Send Alert Emails Every Minute'
)
BEGIN
    EXEC dbo.sp_attach_schedule
        @job_name = N'Send Alert Emails Using Powershell',
        @schedule_name = N'Send Alert Emails Every Minute';
END
GO
```

Step 6 - Testing - Check Query Table and Email Test (Optional)

After everything is setup, this is up to you but it's generally a good idea to test functionality to make sure it's working. You can either run the SQL Job Manually, or have it enabled and on a schedule. But will need that job to run in order for this test to work.

Here is a SQL command you can run to create a message for sending. You can modify the subject and body however you like.

```
EXEC master.dbo.usp_SendAlertEmail
    @Subject = N'SQL Agent Test',
    @Body = N'This came from SQL through the queue.',
    @ThrottleMinutes = 0;
```

After running this, you can check the table by running the following query...

```
SELECT TOP (1000) [AlertID]
      ,[EmailTo]
      ,[EmailSubject]
      ,[EmailBody]
      ,[CreatedDate]
      ,[ProcessedDate]
      ,[Status]
```

```
    ,[Status]
FROM [msdb].[dbo].[SQLAlertEmailQueue]
```

Once the Job has run and PowerShell script has executed, you should get your test email.

Step 7 - Setting up Notifications for SQL Agent Jobs (triggered via Script)

Here are the steps for utilizing the new system within SQL Agent Job scripts that send error notifications from within the script. In the Job Step SQL Script itself, towards the end you will see a section like so...

```
EXEC msdb.dbo.sp_notify_operator
   @name    = N''sqlemailoperator'',
   @subject = @Subject,
   @body    = @body;
```

Replace that whole section with this instead....

```
   EXEC master.dbo.usp_SendAlertEmail
   @Subject = @EmailSubject,
   @Body = @body,
   @To = N'yourdestinationemailaddress',
   @ThrottleMinutes = 5;
```

This will use the custom alert email system rather than the built in Database Mail system and your emails will come through as normal.

Step 8 - Setting up Notifications for SQL Agent Jobs Failures

So for SQL jobs, there is notifications that can be setup in the job scripts, but for actual SQL Agent Job failures, those use system notifications via Database Mail to send alerts that a Job or Step fails within that job, so those will need to be adjusted as well for any jobs you want to alert if the job itself fails.

To do so, you need to make some modifications to the SQL Agent Job steps. First thing you need to do is to add a new step to the job that runs the below T-SQL query and then modify all other steps to point to the new step you created on Failure only. You can name it however you like, but the basic layout is as follows.

Your Job Step runs your current existing script step -> Script or Job itself fails -> Then skips to the new step you created for alerting -> Feeds the alert details to the script you add -> Script then adds an entry into the Mail Queue table created above -> Goes through the process to send an alert email, and sets the Job Status to failed.

So essentially we are leveraging what we created above to send the alert emails. So when you create this new step, set it to Quit Job reporting Failure on both Completion and Failure for this step only, then on every other task before it, set that to skip to the new step only on Failure.

Below is the script to add to your new step, you can change the highlighted email below to whatever email you want notifications sent too...

```
DECLARE @JobName         sysname       = N'$(ESCAPE_SQUOTE(JOBNAME))';
DECLARE @ServerName      sysname       = @@SERVERNAME;
DECLARE @Environment     nvarchar(50)  = N'Production';

DECLARE @FailedStepID     int;
DECLARE @FailedStepName   sysname;
DECLARE @FailedMessage    nvarchar(MAX);
DECLARE @FailedRunDateTime datetime;

SELECT TOP (1)
   @FailedStepID = h.step_id,
   @FailedStepName = h.step_name,
   @FailedMessage = h.message,
   @FailedRunDateTime = msdb.dbo.agent_datetime(h.run_date, h.run_time)
FROM msdb.dbo.sysjobhistory h
INNER JOIN msdb.dbo.sysjobs j
   ON h.job_id = j.job_id
WHERE j.name = @JobName
  AND h.step_id > 0
  AND h.run_status = 0
ORDER BY h.instance_id DESC;
```

```
ORDER BY h.instance_id DESC;

IF @FailedStepID IS NULL
BEGIN
   SET @FailedStepID = -1;
   SET @FailedStepName = N'Unknown';
   SET @FailedMessage = N'Could not retrieve failed step details from msdb.';
   SET @FailedRunDateTime = GETDATE();
END;

DECLARE @Subject nvarchar(500);
DECLARE @Body    nvarchar(MAX);

SET @Subject = CONCAT(
   N'[SQL JOB FAILURE] ',
   @Environment, N' | ',
   @ServerName, N' | ',
   @JobName, N' | Step ',
   CONVERT(nvarchar(20), @FailedStepID)
);

SET @Body = CONCAT(
   N'SQL Agent job failure detected.', CHAR(13), CHAR(10), CHAR(13), CHAR(10),
   N'Environment : ', @Environment, CHAR(13), CHAR(10),
   N'Server      : ', @ServerName, CHAR(13), CHAR(10),
   N'Job Name    : ', @JobName, CHAR(13), CHAR(10),
   N'Failed Step : ', CONVERT(nvarchar(20), @FailedStepID), N' - ', ISNULL(@FailedStepName, N''), CHAR(13), CHAR(10),
   N'Time        : ', CONVERT(nvarchar(30), @FailedRunDateTime, 120), CHAR(13), CHAR(10), CHAR(13), CHAR(10),
   N'Error Message:', CHAR(13), CHAR(10),
   ISNULL(@FailedMessage, N'')
);

EXEC master.dbo.usp_SendAlertEmail
   @Subject = @Subject,
   @Body = @Body,
   @To = N'destinationemailaddress',
   @ThrottleMinutes = 1;
```

And your alert emails on job failures should look similar to this...

```
Subject:
[SQL ALERT] Production | SQLSERVER01 | Job Failure

Body:

SQL Agent job failure detected.

Environment : Production
Server      : SQLSERVER01
Job Name    : Data Load Job
Step        : 3 - Insert Data
Time        : 2026-01-01 10:00:00

Error Message:
Violation of PRIMARY KEY constraint...
```

Step 9 - Setting up Notifications on SQL Agent Alerts

Setting up notifications on SQL Agent Alerts takes a little bit more setup than on SQL Agent Jobs because Alerts are designed to send notifications via Database Mail by default and are built that way. However, you can have the SQL Alerts execute a SQL Job on alert and that's what we will utilize for these types of notifications, but leveraging the same table and SQL/PowerShell system we setup already. Follow these steps in order...

a. **Enable Token Replacement on the SQL Server Agent**

First up you will want to enable "**Replace tokens for all job responses to alerts**" on the SQL Server Agent itself. To do that you need to use SSMS.

Connect to the SQL server you want to change the setting on and right click on the **SQL Server Agent** and select **Properties**.

Connect to the SQL server you want to change the setting on and right click on the **SQL Server Agent** and select **Properties**.

Click on Alert System, then check the box "**Replace tokens for all job responses to alerts**" (if it's not already) and click on OK.

This change does not require a SQL Server Agent restart and takes effect immediately.

b. Setup Stored Procedure to ingest SQL Alerts

We will need to create a Stored Procedure that will take the SQL Alert notifications and format them into our current existing mail table structure for processing. This works hand-in-hand with the SQL job we will create next to do so. Run the following to create that stored procedure.

```
USE [master];
GO

CREATE OR ALTER PROCEDURE dbo.usp_QueueSqlAgentAlert
   @AlertMessage  NVARCHAR(MAX) = NULL,
   @ErrorNumber   INT = NULL,
   @Severity      INT = NULL,
   @DatabaseName  SYSNAME = NULL,
   @ServerName    SYSNAME = NULL,
   @JobName       SYSNAME = NULL,
   @To            NVARCHAR(500) = N'destinationemailaddress',
   @ThrottleMinutes INT = 10
AS
BEGIN
   SET NOCOUNT ON;

   DECLARE
      @Subject   NVARCHAR(500),
      @Body      NVARCHAR(MAX),
      @AlertType NVARCHAR(100),
      @CRLF      NCHAR(2) = CHAR(13) + CHAR(10);

   SET @ServerName = COALESCE(NULLIF(@ServerName, N''), @@SERVERNAME);
   SET @DatabaseName = COALESCE(NULLIF(@DatabaseName, N''), N'(not supplied)');

   SET @AlertType =
      CASE
         WHEN @ErrorNumber IN (823, 824, 825) THEN N'I/O / Corruption Alert'
         WHEN @ErrorNumber = 9002 THEN N'Transaction Log Full'
         WHEN @Severity >= 20 THEN N'Critical SQL Severity Alert'
         WHEN @Severity BETWEEN 17 AND 19 THEN N'High Severity SQL Alert'
         WHEN @ErrorNumber IS NOT NULL THEN N'SQL Error Alert'
         ELSE N'SQL Agent Alert'
      END;

   SET @Subject =
      LEFT
      (
         CONCAT
         (
            @AlertType,
            N' - ',
            @ServerName,
            N' - ',
            @DatabaseName,
            CASE
               WHEN @ErrorNumber IS NOT NULL
                  THEN CONCAT(N' - Error ', CONVERT(NVARCHAR(20), @ErrorNumber))
               ELSE N''
            END
         ),
         500
      );

   SET @Body =
      CONCAT
```

```
      CONCAT
      (
        N'A SQL Agent Alert has fired.', @CRLF, @CRLF,
        N'Server: ', @ServerName, @CRLF,
        N'Database: ', @DatabaseName, @CRLF,
        N'Error Number: ', COALESCE(CONVERT(NVARCHAR(20), @ErrorNumber), N'(null)'), @CRLF,
        N'Severity: ', COALESCE(CONVERT(NVARCHAR(20), @Severity), N'(null)'), @CRLF,
        N'UTC Time: ', CONVERT(NVARCHAR(19), GETUTCDATE(), 120), @CRLF,
        N'Local Time: ', CONVERT(NVARCHAR(19), GETDATE(), 120), @CRLF,
        CASE
          WHEN @JobName IS NOT NULL THEN CONCAT(N'Response Job: ', @JobName, @CRLF)
          ELSE N''
        END,
        @CRLF,
        N'Alert Message:', @CRLF,
        COALESCE(@AlertMessage, N'(no message supplied)')
      );

    EXEC dbo.usp_SendAlertEmail
      @Subject = @Subject,
      @Body = @Body,
      @To = @To,
      @ThrottleMinutes = @ThrottleMinutes;
END;
GO
```

c. Create SQL Job to process alert notifications

For this step we will be creating a SQL job each alert will trigger that works along with the Stored Procedure we created to format and create a pending message in the mail notification table to be processed by the existing SQL/PowerShell mail system we already have set up and in the end an email sent out. This job does not use or need a schedule as it's ran on demand so make sure it's enabled at all times. Runthe following to create the job. After setup, you may want to run a test to make sure this system works. You can follow the directions in Step 9 of this page to do so.

```
USE [msdb];
GO

DECLARE @jobId UNIQUEIDENTIFIER;

IF NOT EXISTS
(
    SELECT 1
    FROM msdb.dbo.sysjobs
    WHERE name = N'Queue SQL Agent Alert'
)
BEGIN
    EXEC msdb.dbo.sp_add_job
      @job_name = N'Queue SQL Agent Alert',
      @enabled = 1,
      @description = N'Shared response job for SQL Agent Alerts. Queues alert notifications into msdb.dbo.SQLAlertEmailQueue via
master.dbo.usp_QueueSqlAgentAlert.',
      @start_step_id = 1,
      @owner_login_name = N'sa',
      @job_id = @jobId OUTPUT;

    EXEC msdb.dbo.sp_add_jobstep
      @job_id = @jobId,
      @step_id = 1,
      @step_name = N'Queue SQL Agent Alert Email',
      @subsystem = N'TSQL',
      @database_name = N'master',
      @command = N'
DECLARE
    @AlertMessage NVARCHAR(MAX) = N''$(ESCAPE_SQUOTE(A-MSG))'',
    @ErrorNumber  INT = TRY_CONVERT(INT, N''$(ESCAPE_NONE(A-ERR))''),
    @Severity     INT = TRY_CONVERT(INT, N''$(ESCAPE_NONE(A-SEV))''),
```

```
    @Severity     INT = TRY_CONVERT(INT, N''$(ESCAPE_NONE(A-SEV))''),
    @DatabaseName SYSNAME = N''$(ESCAPE_SQUOTE(A-DBN))'',
    @ServerName   SYSNAME = N''$(ESCAPE_SQUOTE(A-SVR))'';

EXEC master.dbo.usp_QueueSqlAgentAlert
    @AlertMessage = @AlertMessage,
    @ErrorNumber  = @ErrorNumber,
    @Severity     = @Severity,
    @DatabaseName = @DatabaseName,
    @ServerName   = @ServerName,
    @JobName      = N''Queue SQL Agent Alert'';
',
        @on_success_action = 1,
        @on_fail_action = 2;

    EXEC msdb.dbo.sp_add_jobserver
        @job_id = @jobId,
        @server_name = N'(LOCAL)';
END;
GO
```

d. Set SQL Alerts to use SQL Job for Notifications

Now that you have everything setup, you can then set all to use the new job for notifications. You can do this using SSMS.

In Object Explorer, expand **SQL Server Agent** and then expand **Alerts**. Find the alert you want to update, right click on it and then select **Properties**.

On the Properties page, select **Response**. Then uncheck **Notify Operators** (if it's checked) and check **Execute Job**. Click the drop down box underneath and select the **Queue SQL Agent Alert** job from the list. Click on **OK** when done.

You'll want/need to do this for every SQL Server Alert you want to have notifications sent for existing and future alerts as well.

You can check your progress on what alert has the job set by running this query...

```
USE [msdb];
GO

SELECT
    a.name AS AlertName,
    a.enabled,
    j.name AS CurrentResponseJob,
    a.message_id,
    a.severity,
    a.delay_between_responses
FROM msdb.dbo.sysalerts a
LEFT JOIN msdb.dbo.sysjobs j
    ON a.job_id = j.job_id
ORDER BY a.name;
GO
```

Step 10 - Testing SQL Server Alert Notifications (Optional - But Recommended)

A good practice when setting up a system such as this, it never hurts to test to make sure the system is working as intended. It's not required, but recommended to make sure everything is running as it should. Make sure both the SQL Queue Agenet Alert and PowerShell SQL jobs are running before testing.

To do so, you'll want to create a test alert. All this does is capture Pipeline errors and sends an alert. This doesn't affect anything negatively, it's just for testing. Run this to create the test alert...

```
USE [msdb];
GO

IF NOT EXISTS
(
    SELECT 1
```

```
    SELECT 1
    FROM msdb.dbo.sysalerts
    WHERE name = N'TEST - SQL Alert Queue Pipeline - Severity 16'
)
BEGIN
    EXEC msdb.dbo.sp_add_alert
        @name = N'TEST - SQL Alert Queue Pipeline - Severity 16',
        @message_id = 0,
        @severity = 16,
        @enabled = 1,
        @delay_between_responses = 0,
        @include_event_description_in = 1,
        @notification_message = N'Test severity-based alert to validate SQL Agent alert queue pipeline',
        @job_name = N'Queue SQL Agent Alert';
END;
GO
```

Then run the following query to trigger the alert...

```
RAISERROR(N'Test SQL Agent Alert Pipeline', 16, 1) WITH LOG;
GO
```

If you receive the alert email that indicates the system is working as intended and you should be all set. Feel free to delete the Test alert you created, or can disable it if you want to keep it for further/future testing purposes.